[INSTITUTE NAME]
M.Tech Programme | Data Management for Machine Learning (DM4ML) | Assignment 2

Design and Implementation of an End-to-End Data Management Pipeline for a Movie Recommendation System

Technical Report

Submitted by: [Student Name] (Roll No. [ID])
Code repository: [GitHub URL] | November 2026

Abstract

This report presents a reproducible, modular pipeline that turns the MovieLens 100K ratings dataset into evaluated top-N movie recommendations. The pipeline has seven stages: ingestion, validation, preparation, transformation, feature management, training and evaluation. Apache Airflow orchestrates the stages and MLflow tracks the experiments. A rule-based validation layer backed by Pandera schema contracts detected all 320 deliberately injected corrupt records with zero false positives. A versioned feature store of 31 documented features serves both training and prediction. Three models are compared: a popularity baseline, item-based collaborative filtering (item-kNN) and biased matrix factorization. Under a strict global temporal split, the popularity baseline is competitive (NDCG@10 = 0.153 vs. 0.136). When every user has history (per-user temporal split), item-kNN improves precision@10 by 53% and NDCG@10 by 62% over the baseline. Matrix factorization achieves the lowest rating error (RMSE = 0.979) but the weakest ranking quality.

Keywords: *data pipeline, data validation, feature store, recommender systems, MLOps, Apache Airflow, MLflow*

1. INTRODUCTION

Recommendation quality depends on the data behind it as much as on the model. Production recommenders ingest interaction logs continuously, and those logs contain duplicates, malformed values, late or impossible timestamps and records referring to unknown users or items. Without a disciplined data-management layer, these defects silently degrade the models and make results impossible to reproduce. This project therefore treats the data pipeline, not the algorithm, as the primary deliverable. Its objectives are to:

- **ingest and validate** raw interaction data automatically, quarantining invalid records with traceable reasons;
- **prepare and transform** the data into leakage-free training and test sets and reusable derived datasets;
- **manage features** centrally, with documentation and versioning, so that training and serving use identical definitions;
- **train, track and evaluate** a baseline and personalised models under an orchestrated, reproducible workflow.

The **MovieLens 100K** dataset [1] contains 100,000 explicit ratings (1-5 stars) given by 943 users to 1,682 movies between September 1997 and April 1998, with user demographics and 19 genre flags per movie; the rating matrix is 93.7% sparse.

2. SYSTEM ARCHITECTURE

2.1 Architectural overview

The system follows a **layered, stage-oriented architecture** (Figure 1). Five layers are separated by clear contracts. (i) A *storage layer* of zone folders: raw, errors, processed and features. (ii) A *processing layer* of seven stateless stage modules, each with a single `run()` entry point. (iii) A *feature layer* that materialises and serves features. (iv) A *model layer* with a common recommender interface. (v) A cross-cutting *orchestration and tracking layer* made up of Airflow, MLflow, a central YAML configuration and logging. Stages communicate only through files in the storage zones and never through in-memory state. This makes every task idempotent, independently re-runnable and easy to inspect.

2.2 Component specification

Table 1: Pipeline components, their inputs and outputs

Stage	Module	Input	Output (artefacts)
1. Ingestion	src/ingestion/ingest.py	GroupLens archive / mirror	raw CSVs, ingestion_manifest.json
2. Validation	src/validation/validate.py	raw CSVs	validated parquet, errors/*.csv, validation report
3. Preparation	src/preparation/prepare.py	validated parquet	cleaned entities, train/test, split metadata
4. Transformation	src/transformation/transform.py	train split	activity & popularity summaries, sparse matrix
5. Feature store	src/features/*.py	train split, entities	feature tables, metadata, feature catalog

Stage	Module	Input	Output (artefacts)
6. Training	src/training/train.py, models.py	matrix, item features	model.pkl, model_info.json, MLflow run
7. Evaluation	src/evaluation/*.py	models, test split	comparison table, figure, metrics in MLflow
Serving	src/training/predict.py	model, feature store	top-K list with user profile

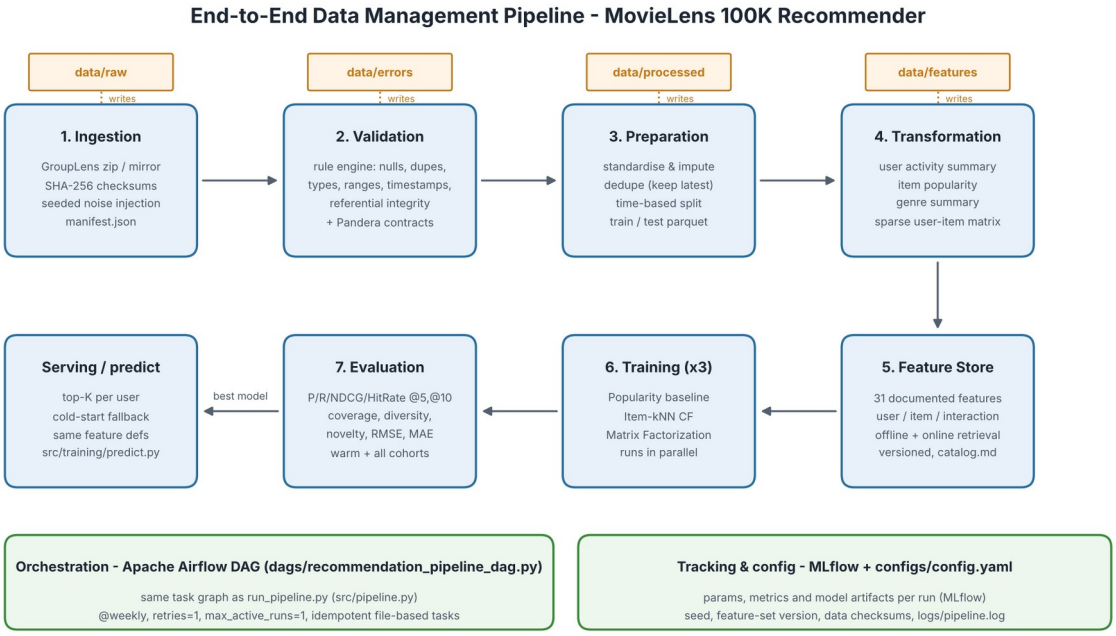


Figure 1: System architecture: storage zones (top), processing stages 1-7 and serving, and the cross-cutting orchestration/tracking layer.

2.3 Data flow and lineage

Table 2 traces record volumes through the pipeline. Every stage writes metadata (row counts, checksums, timestamps), so the provenance of any training example can be reconstructed.

Table 2: Data lineage: record counts at each stage

Checkpoint	Users	Items	Interactions	Note
Source files (u.user, u.item, u.data)	943	1,682	100,000	SHA-256 verified
After ingestion (+ injected defects)	953	1,682	100,410	420 labelled test defects
After validation	943	1,682	100,100	320 rows quarantined
After de-duplication	943	1,682	100,000	latest re-rating kept
Train (to 07-Mar-1998)	751	1,616	80,001	feature/model input
Test (07-Mar to 22-Apr-1998)	303	-	19,999	192 cold-start users

2.4 Orchestration design

The task graph is declared once in `src/pipeline.py` as a mapping from task to its upstream tasks. Two orchestrators are generated from it: the Airflow DAG `movielens_recommendation_pipeline` and a dependency-ordered command-line runner, `run_pipeline.py`, which can be scheduled with cron. The DAG runs weekly with catch-up disabled, one retry per task, a 30-minute timeout and at most one active run, since runs share the storage zones. After feature materialisation the three training tasks fan out in parallel, and evaluation joins them (Figure 2). Tasks return only a small status object to XCom, while large artefacts stay on disk. The DAG was executed successfully on Airflow 3.4 (9/9 tasks, 50.7 s).

3. DESIGN DECISIONS

3.1 Ingestion and data quality

Downloads try the official GroupLens archive first and fall back to a mirror, and files are accepted only if their SHA-256 digests match. Because MovieLens is already clean, the ingestion stage appends a **seeded, labelled set of 420 defective records** covering exact duplicates, re-ratings, nulls, out-of-range ratings (0, 6, 7, 3.5), impossible timestamps (negative or in 2035), orphan IDs, non-numeric strings and invalid users. The labels act as ground truth, so the precision and recall of the validation layer can be measured rather than assumed.

Validation has two tiers. A vectorised **rule engine** applies 24 rules across the three entities. Each rule is either an *error* (the row is quarantined to `data/errors/` with the names of the failed rules) or a *warning* (the row is kept and fixed

downstream, e.g. a missing release date). The surviving rows must then satisfy declarative **Pandera** [6] schema contracts. If a contract fails, the run halts, so later stages can trust their inputs. Raw files are read as strings so that values such as "N/A" are detected as type errors instead of being silently converted to nulls.

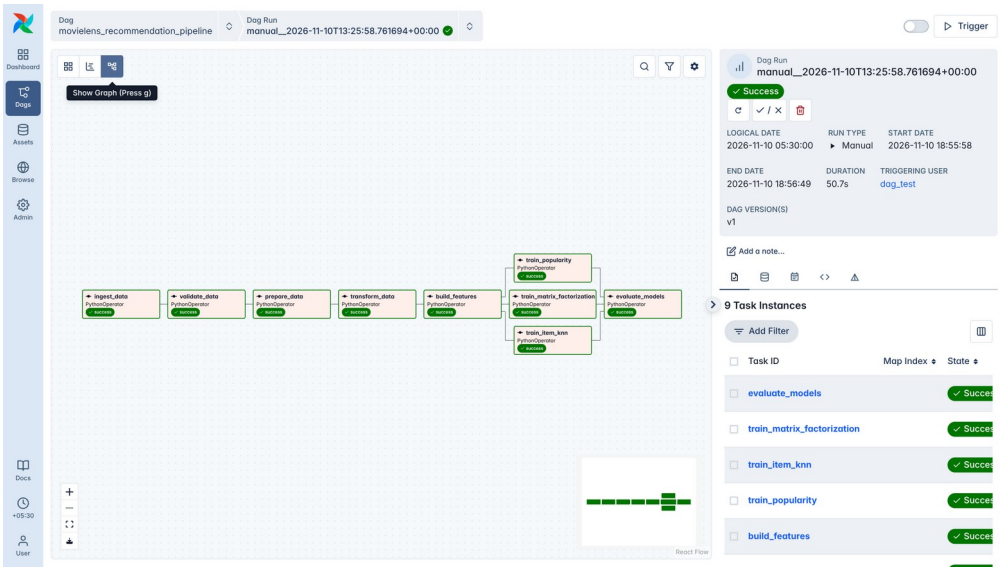


Figure 2: Successful Airflow 3.4 run: linear data-management stages, parallel training fan-out and evaluation join.

3.2 Preparation and temporal splitting

Categorical fields are standardised and the single missing release date is imputed from the title year. The always-empty `video_release_date` column is dropped. Repeated ratings of the same movie by the same user are resolved by keeping the latest one. A random split would leak future behaviour into training, so the primary protocol is a **global temporal split**: all interactions after the 80th-percentile timestamp form the test set, mirroring deployment, where a model trained today serves tomorrow. A **per-user temporal split** (each user's latest 20%) is provided as a configurable sensitivity analysis.

3.3 Feature store

A lightweight, file-based feature store implements the core concepts of Feast without running a server. These are *entities* (user, item, user-item), a *registry* of 31 feature definitions (13 user, 12 item and 6 interaction features, two of them 19-genre groups), offline materialisation with a *point-in-time reference* (the training cutoff), and online retrieval. Each feature is a pure function used both to build the training tables and to recompute a user's features at prediction time, which eliminates training-serving skew. Every materialisation is stamped with a version hash of the definitions and parameters, and the feature catalog (name, entity, type, description, transformation, source, consumer, null rate and statistics) is generated from the registry.

3.4 Recommendation models

- **Popularity baseline:** ranks items by score(i) = $\tilde{r}_i \cdot \log(1 + n_i)$, where $\tilde{r}_i = (n_i \bar{r}_i + m \cdot \mu)/(n_i + m)$ is the Bayesian average with prior weight m = 20. Items with fewer than 10 ratings are excluded.
- **Item-kNN collaborative filtering [3]:** binary cosine similarity between items, pruned to 50 neighbours, ranks the candidates. Ratings are predicted with the kNN-baseline estimator [7] using shrunk, adjusted-cosine similarities.
- **Biased matrix factorization [2]:** $\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$ with 40 latent factors, trained by stochastic gradient descent (30 epochs, learning rate 0.007, L2 penalty 0.06).

All models share one interface (fit / recommend / predict / save) and fall back to the popularity ranking for users without history. Each training task logs its parameters, data version, feature-set version, per-epoch training RMSE, fit time and the serialised model to MLflow [5].

3.5 Evaluation protocol

A test item is relevant if it is rated 4 or higher. For each user, candidates are all catalogue items not rated in training. Ranking accuracy is measured with Precision, Recall, NDCG and Hit-Rate at K = 5 and 10. Beyond-accuracy quality is measured with catalogue Coverage, intra-list genre Diversity and Novelty (mean self-information). Rating accuracy is measured with RMSE and MAE. Users present in training (warm) are reported separately from the full cohort, in which cold-start users receive the fallback list.

4. TOOLS AND TECHNOLOGY STACK

Table 3: Technology choices and rationale

Concern	Technology	Rationale
Storage	CSV (raw), Apache Parquet, SciPy sparse (.npz)	Typed columnar storage; compact CSR matrix for CF
Validation	Custom pandas rule engine + Pandera 0.36	Row-level quarantine plus declarative schema contracts
Feature management	Custom feature store (registry + Parquet)	Feast concepts without infrastructure overhead
Modelling	NumPy / SciPy (own implementations)	Transparent algorithms, few dependencies
Experiment tracking	MLflow 3.18 (SQLite backend)	Parameters, metrics and artefacts per run
Orchestration	Apache Airflow 3.4 + Python runner	Dependencies, retries, scheduling, monitoring UI
Configuration / QA	YAML config, fixed seed, pytest (8 tests)	Reproducibility and verified correctness

5. RESULTS AND DISCUSSION

5.1 Data validation

The rule engine quarantined 310 of 100,410 interaction rows (0.31%) and 10 of 953 user rows. All 320 injected defects were detected and no genuine record was quarantined, giving a precision and recall of 1.0. The 100 injected re-ratings were correctly treated as warnings and resolved during preparation. All three Pandera contracts passed on the surviving data. Table 4 summarises the most frequently triggered rules.

Table 4: Most frequently triggered validation rules

Entity	Rule	Severity	Rows	Handling
interactions	exact_duplicate	error	150	quarantined (first kept)
interactions	repeat_user_item	warning	438	latest rating kept in preparation
interactions	missing_required	error	40	quarantined
interactions	invalid_rating / bad_type_rating	error	40 / 20	quarantined
interactions	invalid_timestamp / orphan user, item	error	30 / 15, 15	quarantined
users	invalid_age, invalid_gender, duplicate_user_id	error	3, 3, 4	quarantined
items	missing_release_date / empty video date	warning	1 / 1,682	imputed / column dropped

5.2 Model performance

Table 5: Global temporal split, warm users (n = 100), K = 10 (best in bold)

Model	P@10	R@10	NDCG@10	HR@10	Cov.	Div.	Nov.	RMSE	MAE
Popularity baseline	**0.123**	**0.102**	**0.153**	**0.460**	0.042	0.679	8.25	1.033	0.836
Item-kNN CF	0.120	0.058	0.136	0.350	0.087	**0.723**	8.46	0.984	0.776
Matrix factorization	0.068	0.070	0.091	0.360	**0.090**	0.714	**9.75**	**0.979**	**0.769**

Table 6: Per-user temporal split (sensitivity analysis), warm users (n = 907), K = 10

Model	P@10	R@10	NDCG@10	HR@10	Cov.	Div.	Nov.	RMSE	MAE
Popularity baseline	0.076	0.065	0.094	0.387	0.045	0.664	8.00	1.093	0.881
Item-kNN CF	**0.116**	**0.138**	**0.152**	**0.565**	0.121	**0.742**	8.24	0.977	0.772
Matrix factorization	0.044	0.039	0.050	0.292	**0.152**	0.728	**9.63**	**0.977**	**0.771**

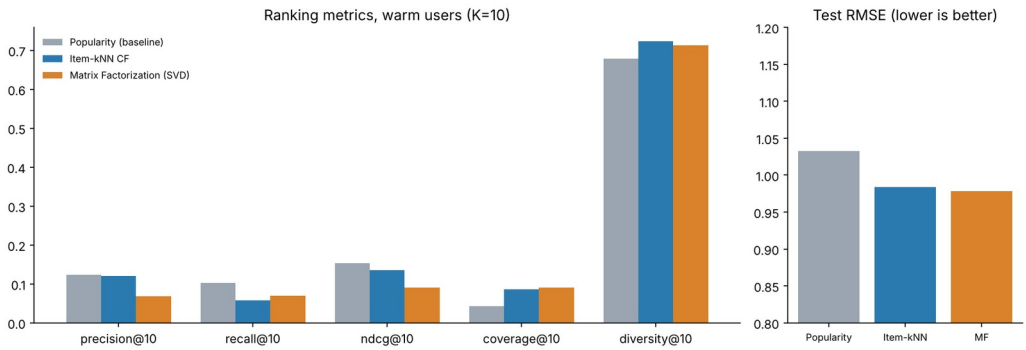


Figure 3: Ranking metrics and test RMSE under the global temporal split (warm users, K = 10).

5.3 Discussion

- **Popularity is a strong baseline under realistic evaluation.** During a short future window, demand concentrates on currently popular titles, and the global split leaves only 100 warm users. The 0.003 precision gap between popularity and item-kNN is within sampling noise. This agrees with Ji et al. [4], who show that global-timeline evaluation narrows or reverses the advantage reported for personalised models.
- **Personalisation pays off once users have history.** In the per-user split, item-kNN beats the baseline on every accuracy metric (+53% P@10, +62% NDCG@10, +46% Hit-Rate). It also recommends 2.7 times more of the catalogue with higher genre diversity, which reduces the popularity bias of the baseline.
- **Good rating prediction does not guarantee good ranking.** Matrix factorization minimises squared error on observed ratings and achieves the lowest RMSE/MAE. However, it scores rarely rated items optimistically, which yields the highest novelty but the lowest top-N accuracy. The training objective must therefore match the product goal (ranking, not rating). The model ordering is unchanged at K = 5.

6. REPRODUCIBILITY AND ENGINEERING PRACTICES

Every parameter lives in one YAML file and a single seed controls noise injection and model initialisation. Repeated full runs, including one from a fresh copy, produced byte-identical evaluation tables. Lineage is captured through the ingestion manifest (source and output checksums), split metadata, a feature-set version hash and MLflow run metadata (parameters, cutoff date, data size and feature version). Eight unit tests cover the metric formulas, the validation rules, leakage-freedom of both splits, the feature computations and the dependency order of the DAG. A complete run takes about 50 s on a laptop; the slowest steps are training item-kNN (18 s) and matrix factorization (21 s).

7. LIMITATIONS AND FUTURE WORK

- **Statistical power:** with 100 warm test users, small differences are not significant. Bootstrap confidence intervals and rolling-origin backtesting would strengthen the comparison.
- **Synthetic defects:** the injected errors follow known patterns. Production data additionally needs distribution-drift monitoring, freshness checks and anomaly detection.
- **Model scope:** matrix factorization is trained on explicit ratings and was not tuned. Implicit-feedback objectives (BPR, weighted ALS) and a hybrid re-ranker would likely improve top-N quality.
- **Cold start:** 63% of test users are new. Demographic features already in the store could drive a content- or demographic-based model for them.
- **Scalability:** Parquet files and SQLite suit a single machine. At scale they would be replaced by Feast with an online store, a database-backed MLflow server with a model registry, and containerised Airflow workers.

8. CONCLUSION

The project delivers a complete, reproducible data-management pipeline for a recommender system. Data are verified at ingestion, validated against both rules and schemas, prepared without temporal leakage, transformed into reusable datasets, served through a documented and versioned feature store, used to train and track three models, and evaluated on accuracy and beyond-accuracy metrics, all orchestrated as an Airflow DAG. Beyond the engineering, the experiments show that conclusions about model quality depend strongly on how the data are split. This underlines the central theme of the course: careful data management is a prerequisite for trustworthy machine learning.

REFERENCES

[1] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," ACM Trans. Interactive Intelligent Systems, vol. 5, no. 4, 2015.
[2] Y. Koren, R. Bell and C. Volinsky, "Matrix factorization techniques for recommender systems," IEEE Computer, vol. 42, no. 8, pp. 30-37, 2009.
[3] B. Sarwar, G. Karypis, J. Konstan and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in Proc. WWW, 2001.
[4] Y. Ji, A. Sun, J. Zhang and C. Li, "A re-visit of the popularity baseline in recommender systems," in Proc. ACM SIGIR, 2020.
[5] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," IEEE Data Engineering Bulletin, vol. 41, no. 4, 2018.
[6] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," in Proc. 19th Python in Science Conf. (SciPy), 2020.
[7] Y. Koren, "Factor in the neighbors: Scalable and accurate collaborative filtering," ACM Trans. Knowledge Discovery from Data, vol. 4, no. 1, 2010.
[8] The Apache Software Foundation, "Apache Airflow documentation," https://airflow.apache.org/docs/, 2026.